8. Accessibility Guide

Contents

Basics 1
`#quote(..)[..]` 1
`tracking` argument 2
Non-breaking space and tilde. 2
Pseudo-move functions 2
  `#move(..)[..]` 2
  `#scale(..)[..]` 2
  `#skew(..)[..]` 2
`#stack(..)` 2
`#math.equation(..)` 3
Artifacts 3
`alt: ".."` argument 3
`lang: ".."` and `region: ".."` 3
Questions 4

Basics

- Don't communicate any information thru visual means only.
- Use elements that carry semantics.
- Remember, that moved elements are read where they are in code.
- Hide decorations from TTS.
- Make document adjustable.

`#quote(..)[..]`

One of the semantically loaded elements. When encountered by TTS, it would be announced as "quote".

"`#quote[..]` – creates an inline quote enclosed in quotation marks."

> `#quote(block: true)[..]` – creates a displayed quote with adjusted margins.

`tracking` argument

Adjust the space between characters. Can carry meaning (e.g., emphasis) for a sighted person, but for TTS assistant won't get it.

`#text(tracking: 0.2em)[SPACED OUT]` ⟶ S P A C E D O U T
`#text(tracking: -0.1em)[squeezed together]` ⟶ squeezed together

Non-breaking space and tilde.

You can (but shouldn't) adjust the space using ~ for non-breaking space. For verbatim ~ use `\~` escaping. IDE can mark this with red color, but it's OK.

Pseudo-move functions

These functions change the dimensions of the wrapped content, but don't affect layout and don't carry semantics. To make them affect the layout, use `reflow: true` argument.

They also place the content in its own block, breaking lines. Use `#box[..]` to avoid this.

`#move(..)[..]`

`#move(dx: len, dy: len)[..]` – shifts the content. The length can be negative.

`#scale(..)[..]`

`#scale(num%)[..]` – changes size of the content.

`#skew(..)[..]`

`#skew(ax: angle, ay: angle)[..]` – \leans *the* content. The angle can be negative (*e.g.*, `-12deg`).

`#stack(..)`

`#stack(dir: .., spacing: len, [..], [..], [..])` puts items one after another in a direction. `dir` argument can take:

`ttb` – top to bottom `btt` – bottom to top `ltr` – left to right `rtl` – right to left

`#math.equation(..)`

Is a full form of `$..$` markup. It takes an equation enclosed in `$`-signs as a positional argument and named argument `block:` that makes it displayed when `true`.

```
#math.equation(
  block: true,
  $2+2=4$
)
```

↓

$$2+2=4$$

Artifacts

Some things aren't intended to be read by TTS (hyphens, headers, page numbers). Such things are called artifacts. Most of them Typst marks automatically. To mark your custom artifact, wrap it into `#pdf.artifact[..]`

`alt: ".."` argument

TTS cannot describe on their own what we see in `#image()`, `#figure()` or `#math.equation()`. That's why these functions take `alt: "str"` argument that will be read by TTS.

`lang: ".."` and `region: ".."`

They are arguments of `#text()` and can be used `#text(lang: "ru")[вызовом функции]` or thru `#set text(..)`. They help TTS read document properly and set names of things, applied typography and spelling check. The `region:` argument specifies a dialect if needed.

There are several special codes for languages:

`"zxx"` – not natural	`"und"` – natural, but unknown	`"mis"` – known, but with no code

Questions

1. Give examples of situations when information is communicated thru visual means only. Why it's bad for accessibility?
2. Give examples of situations, when the document retains the same appearance, but semantic load is lost. Why it's bad for accessibility?
3. How does TTS read elements, whose seen position differs from source-code position?
4. Which element we should hide from TTS "eyes"?
5. Why HTML is better than PDF in terms of adjustability?
6. Explain `#quote()` function.
7. Explain `tracking:` argument.
8. What is ~ used for? How to escape the symbol?
9. What are pseudo-move functions? How to make them affect layout? How to make them not break lines?
10. Explain `#move()` function.
11. Explain `#scale()` function.
12. Explain `#skew()` function.
13. Explain `#stack()` function.
14. Explain `#math.equation()` function.
15. What are artifacts? Explain `#pdf.artifact()` function.
16. Explain `alt:` argument.
17. Explain `lang:` and `region:` arguments.
18. Explain `zxx`, `und` and `mis` language codes.